Multi-Agent Systems – Course Project Report

CLOUT

Campaigning on a Social Network

Consensus with Logic Optimization in Utility driven Targeting in Multi-agent Election Dynamics

Team **Dumbocracy**

Name	Roll number
Pakki Sai Kaushik	AI23BTECH11017
G. Madhava Asrith Murthy	AI23BTECH11007
Vuppula Hemanth Reddy	AI23BTECH11033
Vishank Singh	AI23BTECH11032
B. U. Deekshith Patel	AI23BTECH11003

Under the guidance of **[TODO:** instructor name]

Indian Institute of Technology Hyderabad

November 2026

Code repository: https://github.com/saikaushhikp/CLOUT

Abstract

An election campaign is a contest fought on a network. A party spends money to change what people think, but people also change each other's minds, and the party never sees the electorate directly. We built CLOUT, a single-player election game, to study this situation with the tools of multi-agent systems.

The electorate has between ten thousand and one million simulated voters. They live in four regions and are linked by a friendship network. Each voter wants the government budget split across five sectors in a particular way. The player writes a manifesto (their own budget split) and then has a few campaign rounds to win a majority. Each round they may hire party workers, who start word-of-mouth cascades through the network; buy regional media airings; and pay for survey questions. Meanwhile, every voter drifts towards the average opinion of their friends.

The game connects several course topics in one place. Opinion drift is DeGroot consensus with stubborn agents; party workers spread influence by an Independent Cascade; choosing a manifesto is an optimisation problem on the probability simplex, which we solve with projected gradient and ADMM; and the player's uncertainty about the electorate is described by epistemic logic, with temporal logic describing how a campaign unfolds. We implemented the whole model as one PyTorch code path that runs a single world on a CPU or hundreds of worlds at once on a GPU, together with Bayesian and scripted computer players, a formal-reasoning layer, a web interface for human play, and an experiment harness with paired seeds and machine-checked reporting.

This report describes the game, the electorate, the algorithms and the software. The logic layer and the experiments are reported in Chapters 6 and 7.

Contents

Abstract **i**

1 Introduction **1**
1.1 Overview . . . 1
1.2 Motivation . . . 1
1.3 The game in one paragraph . . . 2
1.4 Questions we asked . . . 2
1.5 Contributions . . . 2
1.6 How this report is organised . . . 3
1.7 Use of AI tools . . . 3
1.8 Who did what . . . 3
1.9 Summary . . . 3

2 Overview of Game Play **4**
2.1 Overview . . . 4
2.2 Phases of a game . . . 4
2.3 Actions . . . 4
2.4 Sequence of a game . . . 5
2.5 Objects we had to build . . . 5
2.6 The voting mechanism . . . 7
2.7 What the player can see . . . 7
2.8 The system update . . . 8
2.9 Design choices and calibration . . . 8
2.10 Summary . . . 9

3 The Electorate **10**
3.1 Overview . . . 10
3.2 Regions . . . 10
3.3 The friendship network . . . 10
3.4 What voters want: archetypes . . . 10
3.5 How voters can be moved: personalities . . . 11
3.6 Zealots . . . 12
3.7 The hidden type of a world . . . 12
3.8 How opinions change . . . 12
3.9 Summary . . . 13

4 Algorithms **14**
4.1 Overview . . . 14
4.2 The worker cascade . . . 14
4.3 Media . . . 15
4.4 Belief over the hidden world . . . 15
4.5 Choosing a survey question . . . 15
4.6 Choosing the manifesto . . . 16

4.7 Planning each round . . . 17
4.8 Reference algorithms . . . 17
4.9 Computer players . . . 18
4.10 Summary . . . 18

5 System Design and Implementation 20
5.1 Overview . . . 20
5.2 Architecture . . . 20
5.3 The game engine . . . 20
5.4 Reproducibility . . . 20
5.5 GPU-first simulation . . . 21
5.6 Keeping hidden information hidden . . . 21
5.7 The web game . . . 22
5.8 Testing . . . 22
5.9 Experiment harness . . . 22
5.10 Summary . . . 22

6 Temporal and Epistemic Logic 24
6.1 Overview . . . 24
6.2 Summary . . . 24

7 Experiments and Strategies 25
7.1 Overview . . . 25
7.2 Summary . . . 25

8 Contributions 26
8.1 Overview . . . 26
8.2 Pakki Sai Kaushik (AI23BTECH11017) . . . 26
8.3 Vuppula Hemanth Reddy (AI23BTECH11033) . . . 26
8.4 G. Madhava Asrith Murthy (AI23BTECH11007) . . . 27
8.5 Vishank Singh (AI23BTECH11032) . . . 27
8.6 B. U. Deekshith Patel (AI23BTECH11003) . . . 27
8.7 Summary . . . 27

9 Conclusion 28
9.1 Overview . . . 28
9.2 What we did . . . 28
9.3 What we learned from building it . . . 28
9.4 Limitations . . . 28
9.5 Future work . . . 29
9.6 Summary . . . 29

References 30

A Notation and Default Parameters 31

B Reproducing the Work 32

List of Figures

2.1 The phases of a game (top) and the order of events inside one campaign round (bottom). 5
2.2 Sequence of one game. Solid arrows are requests, dashed arrows are replies. The player never talks to the simulation core directly. 6
2.3 The main objects of the game. Arrows point from an object to what it holds or uses; the dashed arrow is the allow-listed view that turns the hidden state into a player's observation. 7
2.4 Probability of voting for the party as a function of agreement with the manifesto ($\beta_v = 20$, $z = 0.825$). A voter must agree on more than 82.5% of the budget to lean towards the party. 8

3.1 The friendship network is a stochastic block model whose blocks are the regions. Links are much more likely inside a region than between regions. 11

5.1 Software architecture. Players and the reasoning layer see the game only through observations; the experiment harness drives the simulation core directly for speed. 21

List of Tables

2.1 The player's actions. Media prices scale with region size; 50 coins is the price for a region holding a quarter of the voters. 5

3.1 The public archetype library. Each row is the mean budget split μ of one archetype. 11
3.2 The public personality library. Voters draw their susceptibilities from the Beta distributions of their region's personality. 12

8.1 Division of work among the group members. 26

A.1 Notation used in the report. 31
A.2 Default parameters of the game. 31

List of Algorithms

1 Resolve campaign round t for a player with manifesto b 9
2 Generate an electorate from (N, seed) 12
3 CascadeHop: one round of the worker cascade 15
4 Multi-start projected gradient for the manifesto 16
5 Rollout planning for round t 17
6 Cross-entropy plan search 18

Chapter 1

Introduction

1.1 Overview

This chapter explains what problem we set out to study and why a game is a good way to study it. It gives the real-world motivation, states the questions that drive the project, lists what we built, and describes how the rest of the report is organised. It ends with a short statement of who did what; the full account is in Chapter 8.

1.2 Motivation

Campaigns are expensive, and most of the money goes into changing what people think. The Centre for Media Studies estimated that India's 2019 general election cost between 55,000 and 60,000 crore rupees, and called it the most expensive election ever held. Spending at this scale raises an obvious question: what is the money actually buying, and how should it be spent?

Part of the answer is that opinions move through social ties, not only through advertising. In a randomised experiment on 61 million Facebook users during the 2010 US congressional elections, a message that showed which of a user's friends had voted raised turnout. Most of the effect came from people passing it on to close friends, not from the message itself. A large field experiment by Gerber and Green found that face-to-face canvassing raised turnout far more than phone calls or direct mail. Personal contact, the job of party workers, is one of the strongest tools a campaign has.

The same network that carries a campaign's message can also be exploited. In 2018 *The Guardian* reported that data from about 50 million Facebook profiles had been harvested by Cambridge Analytica to build psychological profiles of voters for targeted political advertising. How much such targeting actually changes votes is still hard to measure, which is exactly why a controlled setting is useful.

Three features of real campaigns stand out from this evidence.

- **Opinions move by themselves.** People talk to friends and slowly converge on what their social circle thinks. Opinion dynamics models, such as DeGroot averaging, describe this drift.
- **Influence spreads like a contagion.** A message passed from person to person can die out or take off. Contagion models, such as the Independent Cascade, describe this, and they have a sharp threshold between the two outcomes.
- **Campaigns work in the dark.** A party does not know exactly what each region wants or how easily it can be persuaded. Every decision is taken under uncertainty, and the questions *"what do I know?"* and *"can I still guarantee a win?"* have precise meanings in epistemic logic.

Experimenting on real elections is neither practical nor ethical. We therefore built a game in which all three features interact. A game has clear rules, a clear goal, and a player who must act with incomplete information. Because we wrote the simulator ourselves, we can also open the box afterwards: replay a game, look at the hidden electorate, and check what the player

could and could not have known. The game is a multi-agent system in the sense of: thousands of simple agents (voters) follow local rules, and one strategic agent (the party) tries to steer the collective outcome.

1.3 The game in one paragraph

CLOUT is a single-player election game. A synthetic electorate of N voters (between 10^3 and 10^6; 10^4 by default) lives in four regions joined by a friendship network. Every voter has a preferred split of a government budget over five sectors: education, welfare, agriculture, industry and the environment. The player begins with a treasury of 500 coins and some free public information. They may pay for survey questions, then they commit to a manifesto: their own five-sector budget split. After that they play five campaign rounds. In each round they may hire party workers in any region, who try to convert their friends, and buy media airings in any region. Between rounds, voters drift towards their friends' views. At the end there is an election, and the player wins with more than half the votes. The regions' true preferences and personalities stay hidden throughout.

1.4 Questions we asked

Q1. Mechanics against theory. Does the simulated electorate behave as course theory predicts? Does consensus speed follow the spectrum of the network, and does the worker cascade show the epidemic threshold that contagion theory predicts?

Q2. Good play. Which strategies win? Does a player that gathers information and plans ahead beat simple rules of thumb, and how close does it come to the best achievable play?

Q3. Information and knowledge. How much is a survey question worth? Can the player learn the hidden personalities of regions? When the game says a win was "guaranteed", how far can that be trusted?

Q4. Game design. Is the game balanced, so that no fixed spending pattern always wins? How much of the outcome is luck, and which design parameters matter most?

1.5 Contributions

1. **A formal model** of a campaign on a networked electorate that combines DeGroot consensus, stubborn agents, an Independent Cascade and a probabilistic voting rule, with an information model that separates what is public, what can be bought and what stays hidden (Chapters 2 and 3).
2. **Decision algorithms** that act only on what a human could see: a Bayesian belief over the hidden world, question selection by information gain, manifesto optimisation on the simplex by projected gradient and consensus ADMM, and planning by batched rollouts (Chapter 4).
3. **A GPU-first implementation**: one PyTorch code path that is bit-identical on the CPU to the reference simulator and runs many worlds in parallel on a GPU, with a web interface and an experiment harness with paired seeds and automatic checking of reported numbers (Chapter 5).
4. **A formal-reasoning layer** for epistemic and temporal properties (Chapter 6) and **a pre-registered experiment suite** of 23 experiments (Chapter 7).

1.6 How this report is organised

Chapter 2 walks through a game: its phases, the actions, the objects we had to build, the voting rule and the algorithm that advances the world each round. Chapter 3 describes how an electorate is generated and how opinions change. Chapter 4 gives the algorithms for the campaign mechanics and for decision making. Chapter 5 describes the software. Chapter 6 covers temporal and epistemic logic, and Chapter 7 the experiments and strategies. Chapter 8 records each member's contribution, and Chapter 9 concludes. The appendices list the notation, the default parameters and the commands needed to reproduce the work. All code is public at https://github.com/saikaushhikp/CLOUT.

1.7 Use of AI tools

We used AI coding assistants while writing parts of the simulator, the tests, the experiment scripts and the plotting code. Every design decision, every rule of the game and every experiment was decided by the group, and all generated code was reviewed and tested before use. The text of this report was written by the group, which takes joint responsibility for all of its content.

1.8 Who did what

In brief (details in Chapter 8):

- **Pakki Sai Kaushik**: project lead and specification; game engine, reproducibility and GPU batching; experiment harness and experiments.
- **Vuppula Hemanth Reddy**: electorate generation, social dynamics and network analysis tools.
- **G. Madhava Asrith Murthy**: campaign mechanics (workers, cascade, media), voting, the information model and calibration.
- **Vishank Singh**: decision algorithms: Bayesian belief, question selection, manifesto optimisation, planning, and the computer players.
- **B. U. Deekshith Patel**: temporal and epistemic logic layer, HTTP API and web client, and testing.

1.9 Summary

Campaigns spend heavily to move opinions that also move on their own through friends, and they do so without seeing the electorate. Field experiments and news reports show that social ties and personal contact are among the strongest levers a campaign has. We built a game that captures these features and used it as a research instrument. The next chapter shows how a game is played.

Chapter 2

Overview of Game Play

2.1 Overview

This chapter walks through one game from start to finish. We describe the phases and the order of events, the actions available to the player and what they cost, the sequence of messages between the player and the game, and the objects we had to design to make all of this work. We then define the voting mechanism and what the player is allowed to see, and give the algorithm that advances the world by one round. The electorate itself is described in Chapter 3 and the detailed mechanics in Chapter 4. All default values are collected in Table A.2.

2.2 Phases of a game

A game has four phases (Fig. 2.1).

1. **Setup.** The player chooses the number of voters N, the number of rounds T (3 to 5 in the web game, default 5), a treasury between 500 coins and 1000 coins, and optionally a seed. The game draws a hidden electorate.
2. **Survey.** The player reads the free public information and may pay for survey questions. The phase ends when the player commits a manifesto b, which is fixed for the rest of the game.
3. **Campaign rounds** $t = 1, \ldots, T$. In each round the player chooses an allocation: how many workers to hire in each region and which regions to air media in. They may also ask more questions. When the allocation is submitted, the world advances by one round.
4. **Election and debrief.** After round T the election is held automatically. The player then sees the hidden truth, a full-information benchmark, and an account of what they knew in each round.

Unspent money is worth nothing at the end, so the player has every reason to spend it.

2.3 Actions

The player has four kinds of action (Table 2.1). Every action is checked by the game before it takes effect. An action that breaks a rule, such as spending more than the treasury holds or committing a budget that does not sum to one, is rejected with an error code. A rejected action changes nothing and costs nothing.

Party workers. A worker is a real voter of the chosen region who joins the party. Their preference is set to the manifesto and never changes again. Each new worker tries once to convert each of their friends, and every voter they convert tries to convert their own friends in the next round. Workers are therefore an investment: a worker hired early reaches further than one hired late.

Media. A media airing reaches a share of the voters in one region and pulls each of them a little towards the manifesto. Repeated airings in the same region have diminishing returns.

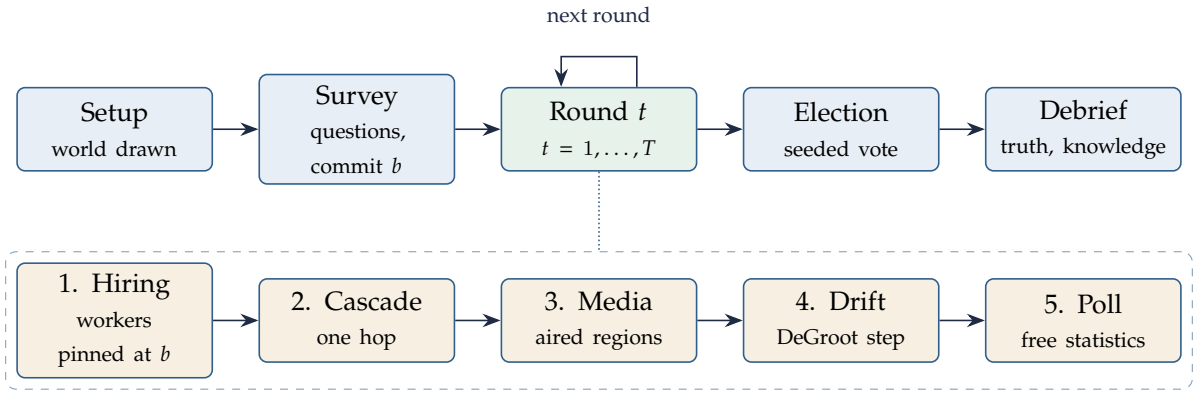


Figure 2.1: The phases of a game (top) and the order of events inside one campaign round (bottom).

Table 2.1: The player's actions. Media prices scale with region size; 50 coins is the price for a region holding a quarter of the voters.

Action	Phase	Cost	Effect
query(s_1, s_2, r)	survey, rounds	10 coins	Share of a fresh 10% sample of region r preferring sector s_2 over s_1, with a margin of error.
commit(b)	survey	free	Fixes the manifesto b for the rest of the game and starts round 1.
allocate(h, m)	rounds	20 coins per worker, plus media	Hires h_r new permanent workers in each region r and airs media in the regions where m_r is true; then the round is resolved.
forfeit	rounds	free	An allocation that buys nothing.

Survey questions. Surveys are the only way to learn about the hidden regions. Each answer is an estimate from a random sample, so it is noisy, and the same question may be asked again for a fresh sample.

2.4 Sequence of a game

Figure 2.2 shows the messages exchanged during one game between the player (a person in a browser, or a computer player), the game engine behind its API, and the simulation core that holds the hidden electorate. The player only ever receives an *observation*: an allow-listed summary that contains no hidden information.

2.5 Objects we had to build

The game needs a small number of carefully separated objects (Fig. 2.3). The separation matters because the whole point of the game is that the player does not see the hidden state.

- **Game configuration**: every rule, price and parameter, plus N, T, the treasury and the seed. It is immutable once the game is created.
- **Electorate**: the hidden world. It holds the friendship graph, the region of each voter, each voter's preference and susceptibilities, the zealots, and the hidden archetype and person-

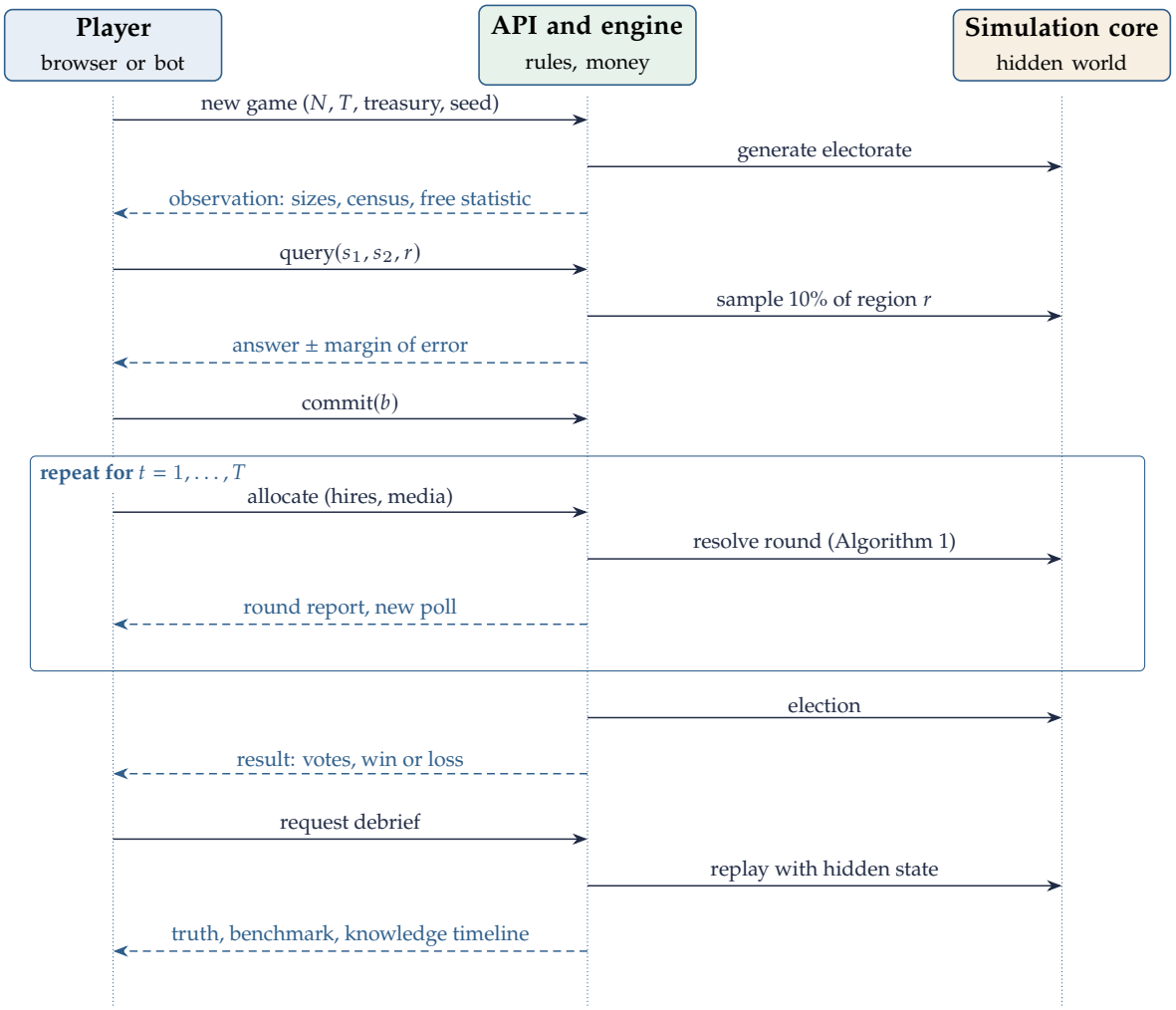


Figure 2.2: Sequence of one game. Solid arrows are requests, dashed arrows are replies. The player never talks to the simulation core directly.

ality maps. It is described in Chapter 3.

- **Game state**: an immutable snapshot of the whole game: phase, round, the current preferences of all voters, one player state per player, the counters of the random streams and an event log. Every accepted action produces a new snapshot.
- **Player state**: the treasury (kept as exact decimals), the manifesto, the workers hired in each region, the number of media airings per region, and the knowledge log.
- **Knowledge log**: an append-only list of everything the player has observed (statistics, survey answers, polls and their own actions), each tagged with its round. It is the basis of the logic layer.
- **Observation**: the player's view, built from the game state by copying only allow-listed public fields. It is the only thing a player, human or computer, ever receives.
- **Action and allocation**: what the player sends back. An allocation lists the workers to hire per region and the regions to air media in.

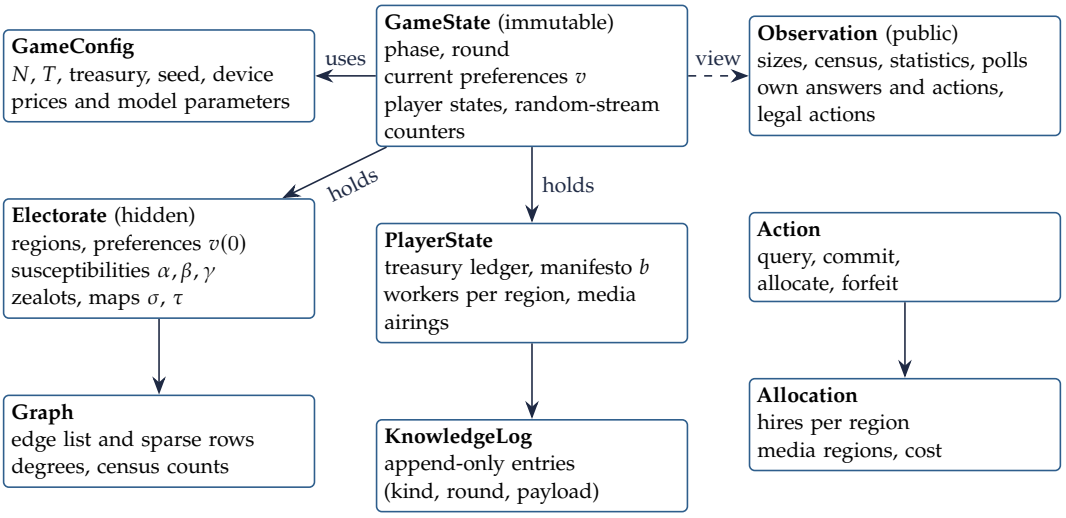


Figure 2.3: The main objects of the game. Arrows point from an object to what it holds or uses; the dashed arrow is the allow-listed view that turns the hidden state into a player's observation.

2.6 The voting mechanism

In the spatial theory of voting, each voter has an ideal point in a space of policies and prefers the platform closest to it. Our voters' ideal points are budget splits, so the space of policies is the probability simplex

$$\Delta = \left\{ x \in \mathbb{R}^5 : x_k \geq 0,\ \textstyle\sum_k x_k = 1 \right\}. \tag{2.1}$$

We measure how far apart two budget splits are by the *total-variation* distance $\mathrm{TV}(x, y) = \frac{1}{2}\|x - y\|_1$: the share of the budget that would have to move to turn one split into the other. Voter i's agreement with the manifesto b is

$$S_i = 1 - \tfrac{1}{2}\,\|v_i - b\|_1 \in [0, 1]. \tag{2.2}$$

Instead of a hard "closest platform wins" rule, a voter supports the party with a probability that rises smoothly with agreement (Fig. 2.4):

$$P_i = \mathrm{sig}\big(\beta_v\,(S_i - z)\big) = \frac{1}{1 + \exp\big(-\beta_v\,(S_i - z)\big)}, \qquad \beta_v = 20, \quad z = 0.825. \tag{2.3}$$

A voter whose agreement equals the threshold z votes for the party half the time, and the slope β_v sets how sharply support rises around it. We call

$$E = \frac{1}{N}\sum_{i=1}^{N} P_i \tag{2.4}$$

the *expected vote share*. It is what the player tries to raise, and the main quantity we analyse. The official result is a seeded random draw in which each voter votes independently with probability P_i. The party wins with more than $N/2$ votes; a tie is a loss.

2.7 What the player can see

The information model separates three kinds of facts.

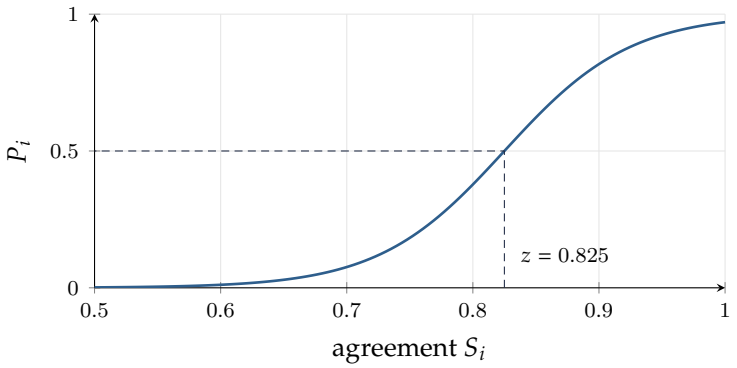


Figure 2.4: Probability of voting for the party as a function of agreement with the manifesto ($\beta_v = 20$, $z = 0.825$). A voter must agree on more than 82.5% of the budget to lean towards the party.

Public at all times.
The rules and prices, N and T, the size of each region, the archetype and personality libraries of Chapter 3 (but not which region has which), the player's own treasury and actions, and a network census: the number of friendships inside each region and between each pair of regions.

Free each round.
For every sector, the share of all voters who rank it in their top three, and a national poll of 400 random voters with its 95% margin of error.

Paid, 10 coins each.
A survey question "in region r, what share of voters prefer sector s_2 over sector s_1?", answered from a fresh random sample of 10% of the region with its margin of error. Repeated answers within a round are also pooled.

Everything else stays hidden: which archetype and personality each region has, the graph, every individual preference and susceptibility, the zealots and all seeds. A world is *consistent* with the player's knowledge log if replaying the player's own actions in that world would have produced the same observations. The set of consistent worlds is the player's *information set*, the starting point of Chapter 6.

2.8 The system update

Algorithm 1 is the heart of the game: what happens when a round's allocation is submitted. Its five steps follow the order of Fig. 2.1. Every random draw comes from a named stream keyed by the round and the player, so the same seed and the same actions always reproduce the same game.

2.9 Design choices and calibration

Several rules above were chosen deliberately, and the default values were tuned before any experiment.

- **Calibration.** Some parameter choices make the game either trivial or hopeless. A calibration study on 200 seeds tuned the voter concentration, the social step, the media strength, and the voting slope and threshold until a uniform manifesto with no campaign rarely won and a well-informed campaign usually did.

Algorithm 1 Resolve campaign round t for a player with manifesto b

Input: state $(v, \text{workers}, \text{converts}_{t-1}, \text{attempts}, \text{airings})$, allocation (h, m)

check that the allocation is valid and affordable; deduct its cost from the treasury
for each region r **do** ▷ 1. hiring
    pick h_r random eligible voters of r (degree ≥ 3, not a zealot, not a worker)
    mark them as workers and set their preference to b
spreaders $\leftarrow$ new workers $\cup$ converts_{t-1} ▷ 2. one cascade hop
$P_j \leftarrow \text{sig}(\beta_v(S_j - z))$ for every voter, from the current preferences
$\text{converts}_t \leftarrow$ CascadeHop(spreaders, P, attempts) ▷ Algorithm 3
set $v_j \leftarrow b$ for every $j \in \text{converts}_t$
$c_i \leftarrow$ media pull on each voter of the aired regions; airings_r += 1 ▷ 3. media
$v \leftarrow$ SocialStep(v, c, b); keep workers at b and zealots fixed ▷ 4. drift, Eq. (3.4)
draw a 400-voter poll and the top-three statistic; log both ▷ 5. poll
if $t = T$ **then** run the election

- **Workers are an investment.** There are no free workers: each worker is bought once, is a real voter of the network, stays pinned at the manifesto, and spreads influence one hop per round. A worker hired early therefore reaches further than one hired late, and the value of a worker depends on the network around it.
- **Prices and choices.** A worker costs 20 coins and a survey question 10 coins; the treasury is the player's choice; and the web game limits T to 3–5 rounds so that games are neither too short to campaign nor so long that drift alone decides them.
- **Noisy surveys.** Survey answers are estimates from a 10% sample rather than exact regional shares, so information is uncertain and repeating a question can be worthwhile.

2.10 Summary

A game moves through setup, a survey phase that ends with committing a manifesto, T campaign rounds and an election. The player can ask survey questions, commit, allocate money to workers and media, or skip a round, and every action is validated before it takes effect. The game is built from a few separated objects: an immutable configuration, a hidden electorate, immutable game states, player states with a knowledge log, and allow-listed observations. Voters vote with a probability that rises steeply once their agreement with the manifesto passes 0.825, and each round is resolved by the five steps of Algorithm 1. The next chapter describes the electorate.

Chapter 3

The Electorate

3.1 Overview

The electorate is the hidden world the player is trying to win over. This chapter describes how it is generated: the regions, the friendship network, what voters want, how easily they can be moved, and the zealots who cannot be moved at all. It then describes how opinions change from round to round, and the theory that predicts how fast. The background on each model is given where it is used.

3.2 Regions

The N voters are split into four regions, North, South, East and West. Every region holds at least 15% of the voters, and the rest are split at random. With $m = \lceil 0.15N \rceil$,

$$\pi \sim \mathrm{Dirichlet}(2,2,2,2), \qquad n_r = m + \left[\mathrm{Multinomial}(N - 4m,\ \pi)\right]_r, \tag{3.1}$$

so $\sum_r n_r = N$ and $n_r \geq m$ for every region. Region sizes are public.

3.3 The friendship network

A social network is a graph whose nodes are people and whose edges are friendships. We write A for its adjacency matrix, d_i for the degree of voter i and D for the diagonal matrix of degrees. The simplest random graph links every pair independently with the same probability, but real societies have more friends inside their own community. The *stochastic block model* captures this by splitting nodes into blocks and letting the link probability depend on the pair of blocks. Our blocks are the four regions (Fig. 3.1).

For each game we draw ten connection weights: one inside each region, $p_r \sim U[0.5, 0.9]$, and one for each pair of regions, $p_{rs} \sim U[0.01, 0.2]$. Used directly they would make the graph almost complete, so all of them are scaled by one sparsity factor s that keeps their ratios but sets the expected mean degree to $\bar{k} = 30$:

$$P_{rr} = s\, p_r, \qquad P_{rs} = s\, p_{rs}, \qquad s = \frac{\bar{k}\, N}{\sum_r p_r\, n_r(n_r - 1) + \sum_{r \neq s} p_{rs}\, n_r n_s}. \tag{3.2}$$

Inside each block every pair of voters is linked independently with probability P. With typical draws about three quarters of a voter's friends live in their own region. Edges are unweighted. The player sees the number of friendships inside and between regions, but never the edges.

3.4 What voters want: archetypes

Each voter i holds a preferred budget split $v_i \in \Delta$ over the five sectors, in the order education, welfare, agriculture, industry, environment. Regions differ in what they want. There is a public

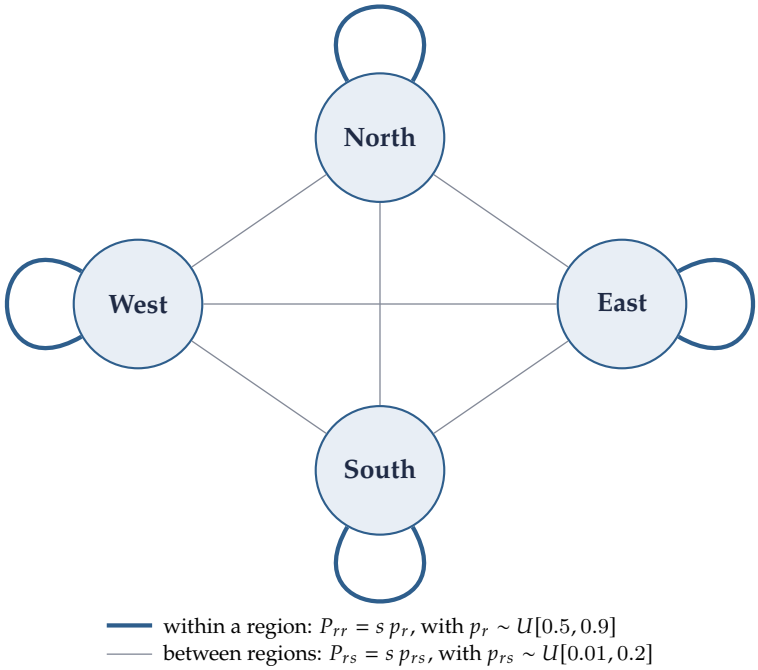


Figure 3.1: The friendship network is a stochastic block model whose blocks are the regions. Links are much more likely inside a region than between regions.

Table 3.1: The public archetype library. Each row is the mean budget split μ of one archetype.

ID	Name	Education	Welfare	Agriculture	Industry	Environment
A0	Agrarian heartland	0.15	0.20	0.40	0.10	0.15
A1	Industrial belt	0.15	0.15	0.10	0.45	0.15
A2	Green urban	0.25	0.10	0.10	0.15	0.40
A3	Welfare-reliant	0.15	0.45	0.15	0.10	0.15
A4	Knowledge hub	0.40	0.10	0.10	0.25	0.15
A5	Balanced middle	0.20	0.20	0.20	0.20	0.20

library of six *archetypes* (Table 3.1), and a hidden one-to-one map σ gives one archetype to each region; there are $6 \cdot 5 \cdot 4 \cdot 3 = 360$ such maps. A voter in region r starts at

$$v_i(0) \sim \text{Dirichlet}\big(\kappa\,\mu_{\sigma(r)}\big), \qquad \kappa = 60, \tag{3.3}$$

so voters of one region want similar but not identical things; a larger κ would make them more alike.

3.5 How voters can be moved: personalities

Voters also differ in how easily they are moved. Each voter has three susceptibilities in $[0, 1]$: to friends (α_i), to media (β_i) and to canvassing by party workers (γ_i). Each region has one of four public *personalities* (Table 3.2), assigned by a hidden permutation τ (24 possibilities), and its voters draw their susceptibilities from that personality's Beta distributions.

Table 3.2: The public personality library. Voters draw their susceptibilities from the Beta distributions of their region's personality.

ID	Name	Friends α	Media β	Canvassing γ
P0	Socially driven	Beta(6,3)	Beta(2,5)	Beta(3,4)
P1	Media-savvy	Beta(3,4)	Beta(6,3)	Beta(2,5)
P2	Independent	Beta(2,6)	Beta(2,6)	Beta(2,6)
P3	Receptive to canvassing	Beta(3,4)	Beta(3,4)	Beta(6,3)

Algorithm 2 Generate an electorate from (N, seed)

derive named random streams (regions, graph, preferences, attributes, zealots) from the seed
draw region sizes n_r with the 15% floor
draw p_r, p_{rs}; compute s; for each block, draw the number of edges, then their endpoints; remove self-loops and duplicates
draw a random archetype map σ and personality map τ
for each voter i in region r **do**
    $v_i(0) \sim \text{Dirichlet}(\kappa\, \mu_{\sigma(r)})$
    $\alpha_i, \beta_i, \gamma_i \sim$ Beta distributions of personality $\tau(r)$
    with probability ζ: make i a zealot ($\alpha_i = \beta_i = \gamma_i = 0$)
return graph, regions, $v(0)$, susceptibilities, zealots, σ, τ

3.6 Zealots

Each voter is independently a *zealot* with probability $\zeta = 0.05$. Zealots never change their mind ($\alpha = \beta = \gamma = 0$), but their friends still listen to them. In opinion dynamics, such *stubborn agents* are known to have outsized effects: a single zealot can pull a whole population towards their view, and zealots who disagree prevent consensus altogether. Our zealots keep the electorate from collapsing to a single opinion, and they cannot be hired or converted.

3.7 The hidden type of a world

The pair $\theta = (\sigma, \tau)$ is the *hidden type* of a world; there are $360 \times 24 = 8640$ of them. Everything else, the graph, the individual preferences, the susceptibilities and the zealots, is the *micro-state*. Survey answers depend only on preferences, which depend only on σ, so surveys can reveal σ but not τ; the player can learn about personalities only indirectly, through polls. Algorithm 2 summarises how a world is generated.

All of this is done with tensor operations, block by block, so memory grows with the number of edges and a million-voter electorate takes a few seconds to build. The electorate is drawn on the CPU and then moved to the GPU, so the same N and seed give exactly the same world on any machine.

3.8 How opinions change

3.8.1 DeGroot averaging

In the DeGroot model, each person repeatedly replaces their opinion with the average of their neighbours' opinions, $x(t+1) = Wx(t)$ with $W = D^{-1}A$. On a connected graph that is not bipartite, opinions converge to a common value. The speed is governed by the second-largest eigenvalue modulus μ_2 of W: disagreement shrinks roughly like μ_2^t. A closely related measure is the algebraic connectivity, the second-smallest eigenvalue of the Laplacian $L = D - A$. A graph with weak links between communities, like ours, has μ_2 close to 1 and converges slowly.

3.8.2 The update rule

Our voters do not average completely each round. Let $a_i = \omega\,\alpha_i$ with $\omega = 0.15$ be the weight voter i gives to friends, and c_i the combined campaign pull on i this round. All voters update at the same time:

$$v_i \leftarrow (1 - a_i - c_i)\,v_i \;+\; a_i\,\bar{v}_{\mathcal{N}(i)} \;+\; c_i\,b, \qquad \bar{v}_{\mathcal{N}(i)} = \frac{1}{d_i}\sum_{j\in\mathcal{N}(i)} v_j, \tag{3.4}$$

where $\mathcal{N}(i)$ is the set of i's friends. If $a_i + c_i$ would exceed 1, the campaign pull is scaled down so that it does not. The new preference is a convex combination of points of the simplex, so it stays in the simplex. Zealots do not update, and party workers are reset to b after every step.

3.8.3 Stubborn agents and resilient averaging

Party workers never change their view, so from the network's point of view they are stubborn agents placed by the party. In the Friedkin–Johnsen model, agents who keep some attachment to a fixed opinion make the population settle at an equilibrium instead of a consensus, and stubborn agents pull that equilibrium towards their own view. Over a long horizon, pinned workers would therefore drag their region towards the manifesto even without any cascade.

Resilient consensus asks how normal agents can still agree when some agents misbehave. In the W-MSR algorithm, each agent discards the F most extreme neighbour values before averaging; it works when the graph is sufficiently *robust*, a property that is hard to check exactly. We implemented W-MSR on the simplex (neighbours are ranked by their TV distance) as an alternative to plain averaging. It acts as a possible defence of the network against the campaign's stubborn workers, and we also compute bounds on the graph's robustness. Bounded-confidence models are another well-known variant, which we did not implement.

3.9 Summary

An electorate is four regions of voters linked by a stochastic block model with mean degree 30 and strong ties inside regions. Each region has a hidden archetype (what it wants) and a hidden personality (how it can be moved), and 5% of voters are zealots. Together the two hidden maps give 8640 world types. Each round voters move part of the way towards the average of their friends, a damped DeGroot step whose speed is set by the spectrum of the network, while workers act as stubborn agents. The next chapter gives the algorithms that act on this electorate.

Chapter 4

Algorithms

4.1 Overview

This chapter gives the algorithms of the game, in two groups. The first group is the campaign mechanics: the worker cascade and media. The second group is decision making: how a player keeps a belief about the hidden world, chooses which question to ask, picks a manifesto, and plans its spending. We end with the reference algorithms we use to judge how well a player does, and with the computer players built from all of these.

4.2 The worker cascade

4.2.1 Background

In the *Independent Cascade* model, when a node becomes active it gets one chance to activate each inactive neighbour j, and succeeds with some probability q_j. Newly activated nodes get their own chance in the next step. Whether the cascade dies out or spreads depends on an *epidemic threshold*: the spectral radius of the adjacency matrix weighted by the transmission probabilities. If each neighbour j converts with probability q_j, the expected number of new activations grows by the operator $A\,D_q$ with $D_q = \mathrm{diag}(q)$, and the cascade grows when $\lambda_{\max}(AD_q) > 1$.

Influence maximisation asks which k seeds give the largest expected spread. Under the Independent Cascade the spread is a *submodular* function of the seed set: each new seed adds less than the previous one. For such functions the greedy algorithm reaches at least $1-1/e$ of the optimum, and the CELF algorithm makes greedy selection much faster by updating marginal gains lazily. Our workers are the seeds of such a cascade.

4.2.2 Our cascade

The spreaders of round t are the workers hired in round t and the voters converted in round $t-1$. Each spreader tries once to convert each friend j (workers and zealots cannot be targeted). If j receives e_j attempts this round, it converts with probability

$$1 - \left(1 - \gamma_j P_j\right)^{e_j}, \tag{4.1}$$

where P_j is j's current probability of voting for the party (Eq. (2.3)). No voter may receive more than three attempts in the whole game, failures included. A convert jumps to $v_j = b$, spreads in the next round only, and is an ordinary voter afterwards. Algorithm 3 gives one hop; it is computed for all voters at once with sparse edge counts, never with a loop over voters.

The factor P_j in Eq. (4.1) is a deliberate choice: voters who already lean towards the manifesto are easier to convert. It also couples the two halves of the game. A manifesto close to what a region wants raises P_j there, which raises $q_j = \gamma_j P_j$ and can push that region above its epidemic threshold $\lambda_{\max}(AD_q) = 1$. The manifesto is not only a policy choice; it also decides how far the party's workers can reach.

Algorithm 3 CascadeHop: one round of the worker cascade

Input: spreaders $\mathcal{S}$, vote probabilities P, attempts used so far u, cap 3

for every voter j that is not a worker or a zealot **do** ▷ done in parallel
  $k_j \leftarrow$ number of edges from $\mathcal{S}$ to j
  $e_j \leftarrow \min(k_j,\ 3 - u_j)$; $u_j \leftarrow u_j + e_j$
  $q_j \leftarrow 1 - (1 - \gamma_j P_j)^{e_j}$
  j converts with probability q_j
return the set of converts

4.3 Media

An airing in region r costs $m_0\, n_r/(N/4)$ with $m_0 = 50$. Each voter of an aired region is reached with probability $p_m = 0.6$, and a reached voter feels a pull towards b of strength

$$c_i = \eta_m(n)\,\beta_i, \qquad \eta_m(n) = \frac{\eta_0}{1 + \lambda n}, \quad \eta_0 = 0.25,\ \lambda = 0.5, \tag{4.2}$$

where n counts the player's earlier airings in that region. The pull enters the update of Eq. (3.4). Repeated airings therefore have diminishing returns, and media helps most in media-savvy regions, where β_i is large.

4.4 Belief over the hidden world

A player who does not know the hidden state faces a partially observable decision problem, which is intractable to solve exactly at our scale. Our players use two standard approximations: a Bayesian belief over hidden worlds, and planning by rollouts.

Survey answers depend only on σ. The belief is therefore an exact posterior over the 360 archetype maps, starting from a uniform prior, while the belief over τ stays uniform. For each archetype, the distribution of answers to each question is precomputed once by Monte Carlo. When an answer $\hat{a}$ arrives, every map is reweighted by a Gaussian likelihood

$$\Pr(\sigma \mid \hat{a}) \ \propto\ \Pr(\sigma)\, \exp\!\Big(-\frac{(\hat{a} - a_\sigma)^2}{2 s_\sigma^2}\Big), \tag{4.3}$$

where a_σ is the answer expected under σ and s_σ combines the sampling error of a 10% sample with the spread of answers across voters. The free top-three statistic is used in the same way.

4.5 Choosing a survey question

Bayesian experimental design suggests asking the question whose answer is expected to reduce uncertainty most. For each of the $10 \times 4 = 40$ possible questions (a pair of sectors and a region) the player computes the mutual information between the answer Y and the map,

$$I(\sigma; Y) = H(\sigma) - \mathbb{E}_Y\big[H(\sigma \mid Y)\big], \tag{4.4}$$

and asks the question with the largest value. It stops when the posterior entropy falls below 0.05 nats, when the best question gains too little per coin, or after ten questions or 50 coins of surveys. In a campaign round it may ask one more question if the gain is large. Repeats are allowed, since each answer is a fresh sample.

Algorithm 4 Multi-start projected gradient for the manifesto

Input: anticipated voters $\{v_i\}$, starts $b_1^{(0)}, \dots, b_M^{(0)}$, step η, smoothing δ

for all starts m in parallel **do**
  repeat
    $b_m \leftarrow P_\Delta\big(b_m + \eta\, \nabla \tilde{f}_\delta(b_m)\big)$ ▷ P_Δ: sort, threshold, clip
  until the change is below 10^{-4} or the iteration limit is reached
return $\arg\max_m f(b_m)$ on the exact, unsmoothed objective

4.6 Choosing the manifesto

4.6.1 The optimisation problem

The manifesto should maximise the expected vote share on election day:

$$\max_{b \in \Delta} \; f(b) = \frac{1}{N} \sum_i \operatorname{sig}\Big(\beta_v \big(1 - \tfrac{1}{2}\|v_i - b\|_1 - z\big)\Big). \tag{4.5}$$

An important lesson from calibration is that the electorate on election day is not the electorate at the start: even without a campaign, social averaging pulls each region towards its mean. Every good player therefore optimises against the *anticipated* electorate, obtained by simulating T rounds of social-only dynamics. A player without full information does this on eight synthetic worlds built from its posterior: each takes a map σ sampled from the belief and draws everything else from the public prior, at a reduced size of 1600 voters that keeps region proportions and degree structure.

The objective is neither smooth, because of the absolute values, nor concave. We replace $|x|$ by the pseudo-Huber function $\sqrt{x^2 + \delta^2} - \delta$ with a small δ, and use several starts: the uniform split, the population mean and the five vertices of the simplex pulled towards its centre. The best start is chosen on the exact objective.

4.6.2 Projected gradient

Projected gradient takes a gradient step and then projects back onto the simplex (Algorithm 4). The Euclidean projection onto the simplex can be computed exactly in $O(n \log n)$ time. It is the *proximal operator* of the simplex's indicator function, so projected gradient is a special case of the proximal gradient method.

4.6.3 Consensus ADMM

ADMM splits a problem into pieces that are solved separately and then forced to agree. We use it to solve Eq. (4.5) as if each region negotiated its own preferred manifesto, the textbook model of distributed optimisation in a multi-agent system. Write $-f(b) = \sum_r f_r(b)$, with f_r region r's share weighted by its size. Each region keeps its own copy x_r, and a shared variable z pulls the copies together:

$$x_r \leftarrow \arg\min_x \; f_r(x) + \tfrac{\rho}{2}\,\|x - z + u_r\|^2, \tag{4.6}$$

$$z \leftarrow P_\Delta\Big(\tfrac{1}{4}\textstyle\sum_r (x_r + u_r)\Big), \tag{4.7}$$

$$u_r \leftarrow u_r + x_r - z. \tag{4.8}$$

Algorithm 5 Rollout planning for round t

Input: observation, belief over σ, number of worlds $B = 8$

$C \leftarrow$ up to 20 valid, affordable candidate allocations
sample B worlds from the belief; in each, replay the player's past manifesto and allocations
for every candidate $a \in C$ and world w, in one batch **do**
    play a in round t, then the even continuation in rounds $t+1, \dots, T$
    $V[a, w] \leftarrow$ expected vote share E on election day
return $\arg\max_a \frac{1}{B} \sum_w V[a, w]$

The first step is each region's local problem, solved inexactly with a few gradient steps. The second is the projection that enforces agreement, and the third accumulates each region's disagreement. Because f_r is not convex, ADMM reaches a stationary point and needs a fairly large penalty ($\rho = 5$) and a smoother surrogate ($\delta = 0.02$) to avoid oscillating. We stop when both the primal residual $\|x - z\|$ and the dual residual fall below 10^{-6}.

4.6.4 Other optimisers

Two more first-order methods are implemented for comparison. *Mirror descent* replaces Euclidean geometry with the entropy, which gives multiplicative updates that stay inside the simplex automatically. *Frank–Wolfe* moves towards the vertex of the simplex that best aligns with the gradient and needs no projection at all.

4.7 Planning each round

Planning uses rollouts: for each candidate action, simulate the rest of the game in several worlds drawn from the belief, follow a fixed default policy afterwards, and pick the action with the best average result (Algorithm 5). The candidate list holds at most twenty allocations built from public information: doing nothing, workers only, media only and mixtures, all valid and affordable. The default continuation is "even" pacing: each later round spends an equal share of the remaining money, half on media in the least-aired regions and half on workers split by region size. All candidates are simulated together in one batched call with common random numbers, so the differences between candidates are not drowned out by noise.

4.8 Reference algorithms

To judge a player we need a target. We use two.

The full-information Oracle. The Oracle is told every voter's true preference and susceptibilities. It optimises the manifesto against the true anticipated electorate, and each round it buys greedily by estimated gain in expected vote share per coin. For media this is the expected change in P_i over the voters who would be reached. For a worker in region r, it estimates the reach of a cascade with per-attempt chance $a_r = \mathbb{E}_r[\gamma P]$ and mean degree $\bar{k}_r$ over the L remaining hops,

$$\text{reach}_r \approx \min\Big(\bar{k}_r a_r \sum_{h=0}^{L-1} (\bar{k}_r a_r)^h,\ \tfrac{1}{2} n_r\Big), \tag{4.9}$$

Algorithm 6 Cross-entropy plan search

Input: true world, manifesto b; 256 plans per iteration, elite fraction 10%, smoothing 0.7

initialise a sampling distribution over per-round worker counts and media choices
for 30 iterations **do**
    sample 256 affordable plans; simulate all of them in one batch
    keep the 26 best plans (the elite)
    refit the distribution to the elite, mixed 0.7 : 0.3 with the previous one
improve the best plan by hill climbing; re-score it on 20 fresh random streams

multiplied by the expected gain per converted voter. It is a sensible heuristic, but nothing guarantees that it is optimal. Its results are always labelled and never mixed with those of fair players.

Plan search. Given a manifesto and the true world, an open-loop search looks for the best whole plan: how many workers and which media in each region, in each round. It uses the cross-entropy method, which repeatedly samples plans, keeps the best fraction and refits the sampling distribution to them (Algorithm 6), followed by hill climbing. The final plan is re-scored on 20 fresh random streams so that its score is not inflated by the noise it was selected on. The search is itself a heuristic, so its score is a lower bound on the best achievable.

4.9 Computer players

The players used in the experiments are built from these algorithms. All except the ORACLE see only the observation a human would see.

- **Bayesian planner (player II)**: belief and question selection, manifesto by projected gradient (or by consensus ADMM in a variant), and rollout planning each round.
- **People Pleaser (player I)**: the same surveys and belief, then a rule of thumb. It picks the smallest group of largest regions that hold a majority, blends their most likely archetypes by size, and spends at an even pace with media in those regions and the rest on workers.
- **Strong-Arm (player III)**: asks nothing, pushes a fixed agenda, and spends half on media and half on workers at one and a half times the even pace, so its money runs out early.
- **Fixed patterns**: *none*, *all workers*, *all media* and *split* (half and half at an even pace), used to check balance.
- **Top-three and uniform**: one uses the free top-three statistic as its manifesto and spends evenly; the other commits the uniform split and never spends.

An interface for a language-model player also exists: it would receive the observation as JSON and reply with one action. It is not implemented yet.

4.10 Summary

The worker cascade is an Independent Cascade with conversion chance $\gamma_j P_j$ and a cap of three attempts per voter, so the manifesto itself moves regions across the epidemic threshold. Media gives a diminishing pull in aired regions. Players keep an exact posterior over the 360 archetype maps, ask the question with the largest expected information gain, optimise the manifesto on the simplex by projected gradient or consensus ADMM against the anticipated electorate, and

choose each round's spending by batched rollouts. A full-information ORACLE and a cross-entropy plan search serve as references. The next chapter describes the software that runs all of this.

Chapter 5

System Design and Implementation

5.1 Overview

This chapter describes the software. We start with the overall architecture, then explain the three properties we cared about most: that every game can be replayed exactly, that the simulator runs fast enough on a GPU to support large experiments, and that the player never sees hidden information. We then describe the web game, the testing strategy and the experiment harness.

5.2 Architecture

Figure 5.1 shows the layers. At the bottom, a *simulation core* implements electorate generation, dynamics, the campaign mechanics, voting and the information model as PyTorch tensor operations. Above it, a *game engine* enforces the rules: phases, validation, money and the order of events in a round. Players and the reasoning layer talk to the engine, the experiment harness drives the simulation core directly, and the web interface reaches the engine over an HTTP API.

The code is a Python package with one module per concern: `electorate`, `dynamics`, `voting`, `information`, `game`, `strategies`, `logic`, `analysis` (spectral and robustness tools) and `api`. The package has about 5,900 lines of Python, the web client about 5,600 lines of TypeScript, and the experiment scripts about 13,000 lines.

5.3 The game engine

The engine is a set of pure functions over immutable game states (Fig. 2.3). Applying an action returns a new state; a rejected action returns the same state object with an error code (for example `INSUFFICIENT_FUNDS` or `INVALID_PHASE`), so a game can never be left half-changed. Money is kept as exact decimals with two places, so prices and treasuries never suffer rounding drift. The engine follows Algorithm 1 and runs the election automatically after the last round.

5.4 Reproducibility

Every random draw comes from a named stream derived from the root seed: `regions`, `graph`, `preferences`, `attributes`, `zealots`, `workers`, `cascade`, `media`, `poll`, `election` and others. Streams are derived with a cryptographic hash, and per-round draws use sub-seeds keyed by stream, round and player. Two consequences follow.

- The same N and seed produce a bit-identical electorate on any machine with the same PyTorch version, so a player can share a seed and someone else can play the same world.
- A game can be replayed exactly from its seed and its list of actions. The reasoning layer relies on this to recover the true state at earlier decision points.

Every experiment result also records its configuration, seeds, git revision, device and library versions.

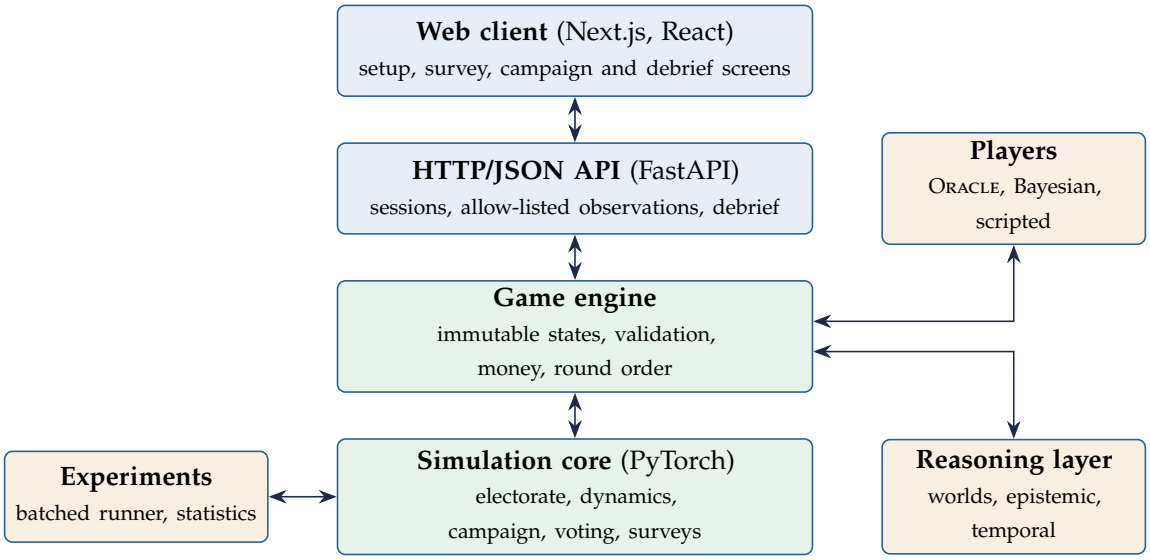


Figure 5.1: Software architecture. Players and the reasoning layer see the game only through observations; the experiment harness drives the simulation core directly for speed.

5.5 GPU-first simulation

All operations on voters are tensor operations; there are no Python loops over voters. The graph is stored as an edge list plus a compressed sparse row view, so memory grows with the number of edges and an $N \times N$ matrix is never formed. Every function in the simulation core accepts an optional leading batch dimension, so the same code simulates one world or B worlds at once.

- **Inside players.** The Bayesian planner joins its eight sampled worlds into one disjoint graph and simulates all candidate plans in one call (Algorithm 5). The reasoning layer does the same.
- **In experiments.** A batched true-world runner plays B complete games per call, each with its own world, manifesto, policy and random streams.

The single code path gives a strong correctness check. On the CPU the batched runner is bit-identical to the reference single-world simulator. On the GPU it produces identical discrete outcomes (who is hired, who converts, who wins), and its floating-point values agree to within 6×10^{-8}. Every experiment ran a three-seed CPU-to-GPU parity check before its main run. Spectral quantities use SciPy's Lanczos solver on the CPU for moderate N and power iteration on the GPU for large N.

5.6 Keeping hidden information hidden

A game about knowledge is only meaningful if the player cannot peek. Observations, API responses, logs and error messages are built by allow-listed serialisers that copy only public fields. A recursive leak test walks every response and fails if it finds any voter-level preference, susceptibility, zealot flag, edge, archetype or personality map, block probability or seed. Computer players receive a deep copy of their own observation, so they cannot reach into the engine either. The post-game debrief is the only place where hidden truth is shown, and only after the game is over.

5.7 The web game

Humans play in a browser. The engine is served by a FastAPI application with routes to create a game, fetch the player's observation, ask a question, commit, allocate, skip a round, and fetch the debrief and knowledge timeline. Each game has its own lock, and idle games are evicted when the server is full. The client is a Next.js application with four screens.

1. **Setup**: choose N, the number of rounds, the treasury and an optional seed.
2. **Survey**: read the free statistics, ask questions, and design the manifesto with sliders and a doughnut chart.
3. **Campaign**: a region map, a sample of the friendship network, a poll chart, the round's intelligence, an allocation panel and an optional round timer.
4. **Results**: the outcome, a round-by-round look back, the hidden truth, the ORACLE benchmark and the knowledge timeline.

The network view draws a stratified sample of voters and the edges between them, since ten thousand nodes would be unreadable and would reveal more structure than the census allows.

5.8 Testing

The repository contains over two hundred automated tests. They cover the statistical properties of the generators (region floors, mean degree, Dirichlet means), invariants of the dynamics (preferences stay on the simplex, workers and zealots never move), the cascade rules (one hop per round, the three-attempt cap), exact money accounting, the phase machine and its error codes, the privacy leak test, the API contract, identities of the logic operators, and CPU-to-GPU agreement. Tests that need a GPU skip automatically without one. The web client has unit tests and browser end-to-end tests.

5.9 Experiment harness

The experiments share a small library. Every arm of an experiment runs on the same seeds (a paired design), and different experiment groups use disjoint blocks of seeds. Results are reported with 95% Wilson intervals for rates and t-intervals for means; paired comparisons use the Wilcoxon signed-rank test for differences in vote share and the exact McNemar test for wins, with the Holm–Bonferroni correction within each experiment. Global sensitivity uses the Morris screening design.

Before each experiment ran, we wrote down what we expected it to show. Each experiment writes raw per-game rows and a summary file, and every figure is redrawn only from those files. A checking script then confirms that every number in the results write-up appears, after rounding, in the data file it cites.

5.10 Summary

The software is a layered Python package on top of PyTorch, with a game engine of pure functions over immutable states, named random streams that make every game replayable, and a single tensor code path that runs one world on a CPU or many on a GPU with verified parity. Allow-listed serialisers and leak tests keep hidden information hidden. A FastAPI server and a Next.js client make the game playable in a browser, over two hundred tests guard the rules, and

a shared harness gives every experiment paired seeds, standard statistics and machine-checked reporting.

Chapter 6

Temporal and Epistemic Logic

6.1 Overview

[**TODO:** To be written by the group.]

6.2 Summary

[**TODO:** To be written by the group.]

Chapter 7

Experiments and Strategies

7.1 Overview

[**TODO:** To be written by the group.]

7.2 Summary

[**TODO:** To be written by the group.]

Chapter 8

Contributions

8.1 Overview

This chapter records who did what. The work was divided into five areas of roughly equal size, one per member, and every member also reviewed the others' work. Table 8.1 gives the overview, and the sections below give the details.

Table 8.1: Division of work among the group members.

Member	Main area	Report
Pakki Sai Kaushik	specification, game engine, reproducibility, GPU batching, experiments	Ch. 1, 5
Vuppula Hemanth Reddy	electorate generation, social dynamics, network analysis	Ch. 3
G. Madhava Asrith Murthy	campaign mechanics, voting, information model, calibration	Ch. 2
Vishank Singh	belief, question selection, optimisation, planning, players	Ch. 4
B. U. Deekshith Patel	logic layer, API and web client, testing	Ch. 6

Chapter 7 (experiments and strategies) and Chapter 9 were written jointly.

8.2 Pakki Sai Kaushik (AI23BTECH11017)

- Project lead: wrote the game specification and the decision register, planned the work, and coordinated reviews.
- Built the game engine: immutable configuration and game states, the phase machine, action validation and error codes, the exact money ledger, and replay.
- Implemented the named random streams that make every game reproducible, and the batched, GPU-first simulation path with its CPU-to-GPU parity checks.
- Built the experiment harness (batched true-world runner, paired seeds, shared statistics, the automatic number checker) and designed and ran the experiment suite.
- Wrote Chapters 1 and 5 of this report.

8.3 Vuppula Hemanth Reddy (AI23BTECH11033)

- Implemented electorate generation (Algorithm 2): region sizes with the 15% floor, the stochastic block model with its sparsity factor, archetype preferences, personalities and zealots.
- Implemented the social dynamics: the damped DeGroot update with pinned workers and frozen zealots, and W-MSR resilient averaging on the simplex.
- Built the network analysis tools: algebraic connectivity, the DeGroot spectral gap, the largest eigenvalue per region, and bounds on graph robustness.
- Wrote Chapter 3 of this report.

8.4 G. Madhava Asrith Murthy (AI23BTECH11007)

- Implemented the campaign mechanics: buying permanent workers, the Independent Cascade that advances one hop per round with a cap of three attempts (Algorithm 3), and media with diminishing returns.
- Implemented voting and the election: agreement, vote probability, expected share and the seeded official vote.
- Implemented the information model: the free top-three statistic, the national poll, sampled survey questions with margins of error and pooling, and the knowledge log.
- Ran the calibration and balance checks that fixed the default parameters and prices.
- Wrote Chapter 2 of this report.

8.5 Vishank Singh (AI23BTECH11032)

- Implemented the Bayesian belief over archetype maps and question selection by expected information gain.
- Implemented the four manifesto optimisers: projected gradient, consensus ADMM, mirror descent and Frank–Wolfe. Also built the anticipated electorate that they optimise against.
- Implemented observation-consistent synthetic worlds, the batched rollout planner (Algorithm 5), the ORACLE, the cross-entropy plan search and the computer players.
- Wrote Chapter 4 of this report.

8.6 B. U. Deekshith Patel (AI23BTECH11003)

- Implemented the logic layer: sampling of worlds consistent with the player's knowledge, the epistemic operators and properties, bounded temporal logic and STL robustness, and the post-game epistemic timeline.
- Built the FastAPI server and the Next.js web client: setup, survey, campaign and results screens, the network view and the epistemic debrief.
- Wrote the privacy leak tests and the API, integration and browser end-to-end tests.
- Wrote Chapter 6 of this report.

8.7 Summary

Sai Kaushik led the project and built the engine and the experiment harness; Hemanth Reddy built the electorate and its dynamics; Madhava Asrith Murthy built the campaign mechanics, voting and the information model; Vishank Singh built the decision algorithms and players; and Deekshith Patel built the logic layer, the web game and the tests. All members share responsibility for the whole report.

Chapter 9

Conclusion

9.1 Overview

This chapter draws the report together: what we set out to do, what we built, what we learned from building it, and where its limits lie.

9.2 What we did

We set out to study an election campaign as a multi-agent system in which a party pushes on an electorate that is already moving, and does so without seeing it. To make that concrete we built CLOUT. Its electorate combines DeGroot consensus, zealots and a stochastic block network. Its party workers are stubborn agents that start an Independent Cascade. Its manifesto is a point on the simplex chosen by projected gradient or ADMM. Its information model makes the player's knowledge a set of possible worlds that can be sampled and reasoned about.

On top of the model we built players that see only what a human sees, a reasoning layer that tells a player after the game what they knew and when the game was decided, a web interface, and a GPU-batched simulator that is exact on the CPU and verified on the GPU. Together these turned the game into an instrument for testing course theory with controlled, paired, pre-registered experiments.

9.3 What we learned from building it

- **The anticipated electorate matters.** Optimising a manifesto against the voters as they are at the start is a mistake, because social averaging moves them before election day.
- **Mechanics and policy are coupled.** Because a voter's conversion chance depends on how much they already like the manifesto, the manifesto changes how far workers' influence spreads. The policy and network halves of the game cannot be designed separately.
- **Knowledge needs care to compute honestly.** Keeping what the player knew separate from what was true required reconstructing earlier observations, a strict privacy boundary and exact replay.
- **One code path pays off.** Writing the simulator once, with a batch dimension, gave us both a reference implementation and a fast one, and made CPU-to-GPU parity a test rather than a hope.

9.4 Limitations

- **A synthetic electorate.** Parameters were tuned so that the game is playable, not fitted to any real electorate. Our conclusions are about this model, not about real elections.
- **One party, one fixed manifesto, a static network.** There is no opponent, the manifesto cannot be revised, friendships never change, and turnout is modelled only as abstention.

- **Heuristic references.** The ORACLE and the plan search are strong heuristics, not proven optima.
- **Sampled knowledge.** Logic verdicts are evaluated over finite samples of worlds and plans at a reduced number of voters.
- **No human data.** All conclusions about play come from computer players; a planned human playtest was dropped for lack of time.

9.5 Future work

- A human playtest of the web game, to compare people with the computer players and to test whether the debrief helps them play better.
- A second party with a multi-party voting rule, to see whether competing manifestos converge as spatial voting theory predicts.
- The language-model player, whose interface already exists.
- Certified bounds on knowledge for small electorates, in place of sampled verdicts.
- Richer dynamics, such as bounded confidence, changing networks and weighted friendships.

9.6 Summary

We built a game that brings consensus, contagion, optimisation, planning and logic together around one model, and used it to study campaigning on a social network under uncertainty. The model is synthetic and single-party, and its references are heuristics, but every part of it is open and reproducible. The code is public at https://github.com/saikaushhikp/CLOUT.

Bibliography

[1] Robert M. Bond, Christopher J. Fariss, Jason J. Jones, Adam D. I. Kramer, Cameron Marlow, Jaime E. Settle, and James H. Fowler. A 61-million-person experiment in social influence and political mobilization. *Nature*, 489(7415):295–298, 2012.

[2] Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011.

[3] Carole Cadwalladr and Emma Graham-Harrison. Revealed: 50 million Facebook profiles harvested for Cambridge Analytica in major data breach. The Guardian, March 2018.

[4] Centre for Media Studies. Poll expenditure: The 2019 elections. Technical report, Centre for Media Studies, New Delhi, 2019.

[5] Morris H. DeGroot. Reaching a consensus. *Journal of the American Statistical Association*, 69 (345):118–121, 1974.

[6] Ronald Fagin, Joseph Y. Halpern, Yoram Moses, and Moshe Y. Vardi. *Reasoning About Knowledge*. MIT Press, 1995.

[7] Noah E. Friedkin and Eugene C. Johnsen. Social influence and opinions. *Journal of Mathematical Sociology*, 15(3–4):193–206, 1990.

[8] Alan S. Gerber and Donald P. Green. The effects of canvassing, telephone calls, and direct mail on voter turnout: A field experiment. *American Political Science Review*, 94(3):653–663, 2000.

[9] David Kempe, Jon Kleinberg, and Éva Tardos. Maximizing the spread of influence through a social network. In *Proceedings of the 9th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 137–146, 2003.

[10] Heath J. LeBlanc, Haotian Zhang, Xenofon Koutsoukos, and Shreyas Sundaram. Resilient asymptotic consensus in robust networks. *IEEE Journal on Selected Areas in Communications*, 31(4):766–781, 2013.

Chapter A

Notation and Default Parameters

Notation

Table A.1: Notation used in the report.

Symbol	Meaning
N, T	number of voters; number of campaign rounds
r, n_r	region index (North, South, East, West); number of voters in region r
Δ	the probability simplex of budget splits over five sectors
$v_i \in \Delta$	voter i's preferred budget split (hidden)
$b \in \Delta$	the player's manifesto
σ, τ	hidden maps from regions to archetypes and to personalities
$\theta = (\sigma, \tau)$	hidden type of a world (8640 possibilities)
$\alpha_i, \beta_i, \gamma_i$	susceptibility to friends, media and canvassing
S_i	agreement with the manifesto, $1 - \frac{1}{2}\|v_i - b\|_1$
P_i	probability that voter i votes for the party
E	expected vote share, the mean of P_i
A, D, L	adjacency, degree and Laplacian matrices of the friendship graph
sig	the logistic function $1/(1 + e^{-x})$

Default parameters

Table A.2: Default parameters of the game.

Parameter	Symbol	Default
Number of voters	N	10^4 (allowed 10^3 to 10^6)
Campaign rounds	T	5 (web game 3 to 5)
Treasury	–	500 coins (allowed 500 to 1000)
Minimum region share	–	15%
Mean degree	$\bar{k}$	30
Within-region link weight	p_r	$U[0.5, 0.9]$
Between-region link weight	p_{rs}	$U[0.01, 0.2]$
Dirichlet concentration	κ	60
Zealot fraction	ζ	0.05
Social step	ω	0.15
Worker price	–	20 coins, paid once
Worker minimum degree; attempt cap	–	3; 3 per voter per game
Media base price	m_0	50 coins per quarter-sized region
Media reach; strength; decay	p_m, η_0, λ	0.6; 0.25; 0.5
Vote slope; threshold	β_v, z	20; 0.825
Survey price; sample fraction	–	10 coins; 10% of the region
Free poll size	–	400 voters

Chapter B

Reproducing the Work

All code is in the public repository https://github.com/saikaushhikp/CLOUT. The commands below assume Python 3.11 or newer and, for the web game, Node.js 22.

Install and test.

```
pip install -e '.[web,analysis,test]'
python -m pytest -q          # GPU tests skip without CUDA
```

Play the game.

```
scripts/run_game.sh          # engine on :61008, web client on :61007
```

Run an experiment and redraw the figures. Each experiment is one script in the experiments folder, for example x6_tournament.py for the strategy tournament. The exact command line of every run is stored in its result file, in the field provenance.command.

```
python experiments/x6_tournament.py --device cuda:0
python experiments/x_report_check.py
```

The second command checks every number in the results write-up against the data file it cites. The figures are redrawn from the saved results by the plotting scripts in the same folder.